

AI Hardware Architecture

Author: Nitin Sarangdhar

Affiliation: SemiX Visiting Professor of practice | IIT Bombay

Date: March 2026

✓ Lab: From Pixels to Predictions (A Hardware-Centric CNN Pipeline)

Objective:

In this lesson, we break down the **Inference Pipeline** of a Convolutional Neural Network (CNN). Unlike the fully-connected XOR network, a CNN uses **Spatially-Aware** operations to detect patterns like edges before making a final classification.

The 5 Stages of the AI Hardware Pipeline:

1. **Convolution (Feature Extraction):** - A 3×3 **Kernel** (Filter) slides over the 10×10 input image.
 - **Hardware Insight:** This is a series of **Multiply-Accumulate (MAC)** operations. In an NPU/TPU, we aim for high "Data Reuse" to minimize power-hungry memory reads.
2. **ReLU Activation (Non-Linearity):**
 - We apply $f(x) = \max(0, x)$ to "clean" the feature map.
 - **Hardware Insight:** This is the "cheapest" operation in silicon (a simple sign-bit check). It introduces **Sparsity**, allowing hardware to potentially skip zero-value computations in later stages.
3. **Max Pooling (Spatial Compression):**
 - We reduce the 8×8 feature map to a 4×4 grid by taking the maximum value in 2×2 neighborhoods.
 - **Hardware Insight:** Pooling reduces the **Memory Footprint** and makes the system robust to small shifts in the input (Spatial Invariance).
4. **Flattening (The Memory Bridge):**
 - We transition from a 2D "Spatial Map" to a 1D "Feature Vector."
 - **Hardware Insight:** This is a **Memory Management** step. We stop using 2D (Row/Col) addressing and treat the SRAM as a contiguous linear buffer for high-speed streaming.
5. **Classification (The 'Brain' Step):**
 - We perform a **Dot Product** between our 1D Feature Vector and a learned **Weight Template**.
 - **Hardware Insight:** This is a **Vector-Matrix Multiplication (VMM)**. The "Winner" (highest score) represents the network's final decision.

Class Discussion: Why do we use a 3×3 kernel instead of a 10×10 kernel? *Answer: Weight sharing! A 3×3 kernel uses only 9 parameters regardless of image size, drastically reducing the transistor count and power leakage on-chip.*

The CNN classifies an input image as a vertical bar, a horizontal bar or neither of the two.

```
1 import numpy as np
2
3 # --- 1. SETUP THE WORLD ---
4 image = np.zeros((10, 10))
5 image[:, 4:6] = 10 # A bright vertical bar (intensity 10)
6
7 kernel = np.array([
8     [-1, 0, 1],
9     [-1, 0, 1],
10    [-1, 0, 1]
11 ])
12
13 # --- 2. DATA OVERVIEW ---
14 print("="*60)
15 print("STEP 1: THE INPUT DATA & KERNEL")
16 print("="*60)
17 print("\nFULL 10x10 INPUT IMAGE:")
18 for row in image:
19     print(" ".join(f"{int(val):3d}" for val in row))
20
```

```

21 print("\n3x3 CONVOLUTION KERNEL (The 'Filter'):")
22 for row in kernel:
23     print(" ".join(f"{int(val):3d}" for val in row))
24
25 print("\n" + "="*60)
26 print("STEP 2: THE SLIDING WINDOW (CONVOLUTION IN ACTION)")
27 print("="*60)
28
29 # --- 3. HELPER FOR REPETITIVE PRINTING ---
30 def show_conv_step(row_idx, col_idx, img, k):
31     # Extract the window
32     window = img[row_idx:row_idx+3, col_idx:col_idx+3]
33     # Multiply
34     multiplied = window * k
35     # Sum
36     result = np.sum(multiplied)
37
38     print(f"\n[STEP {col_idx+1}] SLIDING TO COLUMN {col_idx} (Output Position: {row_idx},{col_idx})")
39     print("-" * 55)
40     print(f"3x3 Image Window at [{row_idx}:{row_idx+3}, {col_idx}:{col_idx+3}]:")
41     print(window)
42     print("\nMultiplied by Kernel (Window * Kernel):")
43     print(multiplied)
44     print(f"--> FINAL SUM (MAC Result) = {result}")
45
46 # --- 4. THE THREE STEPS OF THE SLIDE ---
47 show_conv_step(0, 0, image, kernel)
48 show_conv_step(0, 1, image, kernel)
49 show_conv_step(0, 2, image, kernel)
50
51 # --- 5. THE FINAL RESULT ---
52 def simple_conv(img, k):
53     h, w = img.shape
54     kh, kw = k.shape
55     res = np.zeros((h-kh+1, w-kw+1))
56     for r in range(h-kh+1):
57         for c in range(w-kw+1):
58             res[r, c] = np.sum(img[r:r+kh, c:c+kw] * k)
59     return res
60
61 feature_map = simple_conv(image, kernel)
62
63 print("\n" + "="*60)
64 print("STEP 3: THE FINAL FEATURE MAP (RESULT OF ALL SLIDES)")
65 print("="*60)
66 for row in feature_map:
67     print(" ".join(f"{int(val):3d}" for val in row))

```

```

=====
STEP 1: THE INPUT DATA & KERNEL
=====

```

```

FULL 10x10 INPUT IMAGE:
0  0  0  0  10 10  0  0  0  0
0  0  0  0  10 10  0  0  0  0
0  0  0  0  10 10  0  0  0  0
0  0  0  0  10 10  0  0  0  0
0  0  0  0  10 10  0  0  0  0
0  0  0  0  10 10  0  0  0  0
0  0  0  0  10 10  0  0  0  0
0  0  0  0  10 10  0  0  0  0
0  0  0  0  10 10  0  0  0  0
0  0  0  0  10 10  0  0  0  0

```

```

3x3 CONVOLUTION KERNEL (The 'Filter'):
-1  0  1
-1  0  1
-1  0  1

```

```

=====
STEP 2: THE SLIDING WINDOW (CONVOLUTION IN ACTION)
=====

```

```

[STEP 1] SLIDING TO COLUMN 0 (Output Position: 0,0)
-----
3x3 Image Window at [0:3, 0:3]:
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]

```

```

Multiplied by Kernel (Window * Kernel):
[[-0.  0.  0.]
 [-0.  0.  0.]
 [-0.  0.  0.]]

```

```
--> FINAL SUM (MAC Result) = 0.0

[STEP 2] SLIDING TO COLUMN 1 (Output Position: 0,1)
-----
3x3 Image Window at [0:3, 1:4]:
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]

Multiplied by Kernel (Window * Kernel):
[[-0. 0. 0.]
 [-0. 0. 0.]
 [-0. 0. 0.]]
--> FINAL SUM (MAC Result) = 0.0

[STEP 3] SLIDING TO COLUMN 2 (Output Position: 0,2)
-----
3x3 Image Window at [0:3, 2:5]:
[[ 0. 0. 10.]
 [ 0. 0. 10.]
 [ 0. 0. 10.]]
```

```
1 # --- 6. APPLY ReLU (Rectified Linear Unit) ---
2 # Convention: Apply ReLU immediately after Convolution to remove negative "noise"
3 relu_feature_map = np.maximum(0, feature_map)
4
5 print("\n" + "="*60)
6 print("STEP 4: ReLU ACTIVATION (Removing Negative Values)")
7 print("="*60)
8 print("Notice how the -30s (falling edges) are now 0:")
9 for row in relu_feature_map:
10     print(" ".join(f"{int(val):3d}" for val in row))
11
12 # --- 7. MAX POOLING (2x2 Window, Stride 2) ---
13 def apply_max_pooling(f_map):
14     h, w = f_map.shape
15     pooled_output = np.zeros((h // 2, w // 2))
16     for r in range(0, h, 2):
17         for c in range(0, w, 2):
18             window = f_map[r:r+2, c:c+2]
19             pooled_output[r // 2, c // 2] = np.max(window)
20     return pooled_output
21
22 pooled_result = apply_max_pooling(relu_feature_map)
23
24 print("\n" + "="*60)
25 print("STEP 5: MAX POOLING (8x8 -> 4x4 Compression)")
26 print("="*60)
27 for row in pooled_result:
28     print(" ".join(f"{int(val):3d}" for val in row))
29
30 # --- 8. FLATTENING ---
31 # Converting the 2D grid into a 1D vector for the final classifier
32 final_vector = pooled_result.flatten()
33
34 print("\n" + "="*60)
35 print("STEP 6: FLATTENING (The Final 1D Vector)")
36 print("="*60)
37 print(f"Vector Length: {len(final_vector)}")
38 print("Vector Content:")
39 print(final_vector)
```

```
=====
STEP 4: ReLU ACTIVATION (Removing Negative Values)
=====
Notice how the -30s (falling edges) are now 0:
 0  0 30 30  0  0  0  0
 0  0 30 30  0  0  0  0
 0  0 30 30  0  0  0  0
 0  0 30 30  0  0  0  0
 0  0 30 30  0  0  0  0
 0  0 30 30  0  0  0  0
 0  0 30 30  0  0  0  0
 0  0 30 30  0  0  0  0

=====
STEP 5: MAX POOLING (8x8 -> 4x4 Compression)
=====
 0 30  0  0
 0 30  0  0
 0 30  0  0
 0 30  0  0
```

```
=====
STEP 6: FLATTENING (The Final 1D Vector)
=====
Vector Length: 16
Vector Content:
[ 0. 30.  0.  0.  0. 30.  0.  0.  0. 30.  0.  0.  0. 30.  0.  0.]
```

```
1 import numpy as np
2
3 # --- 1. THE DATA (From your previous Pooling/Flattening step) ---
4 # This is the 1D vector: [0, 30, 0, 0, 0, 30, 0, 0, 0, 30, 0, 0, 0, 30, 0, 0]
5 # (Assuming ReLU was applied before pooling)
6 final_vector = np.array([0, 30, 0, 0, 0, 30, 0, 0, 0, 30, 0, 0, 0, 30, 0, 0])
7
8 # --- 2. THE LEARNED WEIGHTS (Templates) ---
9 # We define what the 'Ideal' Vertical and Horizontal bars look like in 1D.
10 # In a real CNN, the network "learns" these numbers through backpropagation.
11
12 # Vertical Template: High weights where the 2nd column of a 4x4 grid would be
13 # Indices: 1, 5, 9, 13
14 v_weights = np.array([
15     0, 1, 0, 0,
16     0, 1, 0, 0,
17     0, 1, 0, 0,
18     0, 1, 0, 0
19 ])
20
21 # Horizontal Template: High weights where a middle row would be
22 # Indices: 4, 5, 6, 7
23 h_weights = np.array([
24     0, 0, 0, 0,
25     1, 1, 1, 1,
26     0, 0, 0, 0,
27     0, 0, 0, 0
28 ])
29
30 # --- 3. THE INFERENCE (The 'Brain' Step) ---
31 # We calculate the Dot Product: Sum(Vector * Weights)
32 v_score = np.dot(final_vector, v_weights)
33 h_score = np.dot(final_vector, h_weights)
34
35 print("="*60)
36 print("FINAL STEP: CLASSIFICATION VIA DOT PRODUCT")
37 print("="*60)
38 print(f"Input Vector: {final_vector}")
39 print("-" * 60)
40 print(f"Vertical Template Match Score: {v_score}")
41 print(f"Horizontal Template Match Score: {h_score}")
42 print("-" * 60)
43
44 # --- 4. THE DECISION ---
45 if v_score > h_score:
46     print("RESULT >>> VERTICAL EDGE DETECTED")
47 elif h_score > v_score:
48     print("RESULT >>> HORIZONTAL EDGE DETECTED")
49 else:
50     print("RESULT >>> NO CLEAR PATTERN DETECTED")
```

```
=====
FINAL STEP: CLASSIFICATION VIA DOT PRODUCT
=====
Input Vector: [ 0 30  0  0  0 30  0  0  0 30  0  0  0 30  0  0]
-----
Vertical Template Match Score: 120
Horizontal Template Match Score: 30
-----
RESULT >>> VERTICAL EDGE DETECTED
```

